

折返治理 · Switchback

Switchback Governance — the Human-in-the-Loop Governance Layer for Multi-Agent Teams

让每一次 Agent 决策，都沿轨道可查、可停、可回头。

Every agent decision must be traceable, stoppable, and reversible on the rail.

赛道：① 新智基座 Agent Infra —— 多 Agent 分工协同、可复用、可观测的端到端任务闭环

参赛主体：sukikeeling (AI Agent 协作参赛，个人/≤3 人组队)

项目仓库：github.com/sukikeeling/switchback (Apache-2.0，零依赖可运行)

旗舰证据：百年京张 AI 创新带开源征集 84/100 最高纪录 (open-city-ai/haidian)

日期：2026-08-13 | 初赛：方案设计 (本 PDF + 代码包可选)

一、场景与价值 (Why)

1.1 问题：Agent 从 Demo 到 Production 的四道坎

多 Agent 系统跑通 demo 很容易，上生产最难的不是"能不能跑通"，而是四道治理鸿沟：

鸿沟	生产症状	本方案应答
结果不可信	数值三处对不齐、引用错位、模型幻觉	证据核验 Skill：数值三处对齐 + 引用双向 + 内容哈希
过程不可停	编排一旦开始就自动往下走，错误被一路放大	折返点：到站必停，复核后才继续
出事不可回	没有审批/回滚/隔离语义，只能“硬杀进程”	道岔三态：正线 / 侧线折返 / 入段检修，不设自动恢复
账目不可审	谁做了什么、基于哪个数据版本、结论是什么，无账本	K 标账本：内容寻址哈希链，逐条可复核

1.2 真实场景：一场真实竞赛的 16 轮治理实验

本方案不是概念 Demo，而是一段**真实任务闭环**的产品化：参赛主体 sukikeeling 参加了"百年京张 AI 创新带国际城市设计开源征集"（海淀区主办，open-city-ai/haidian），面对确定性 CI 门禁与官方多模态 AI 评审（CocoSgt，100 分制），16 个 PR 中亲历全部治理事故，最终以 **84/100 拿下最高纪录**：

- package_id 残留 —— 长会话状态漂移，v8 旧值混入新版本；
- CRLF 哈希错位 —— 证据链断裂，逐文件哈希核对发现换行符差异；
- 中英标记漂移 42 处 —— 双语文档一致性失控；
- 数值三处对不齐 —— 面积/指标在不同章节复算结果不同。

为什么要做“治理层”

这些事故的根因不是某个 Agent 能力不够，而是**缺少“到站必停”的制度**。京张项目最终的高分配方——原创概念 + 克制表达 + 结构化证据 + 折返复核——正是把踩过的坑制度化为协议。这个协议对任何多 Agent 生产系统都可复制：

零人工运维 智能客服

软件研发协同

金融风控理赔

1.3 行业可复制性

赛道参考方向	折返治理落点
零人工运维（告警→根因→修复→恢复→复盘）	每次修复走 evidence-verify + 折返点；回滚即“侧线折返”
智能客服自主闭环	意图分级即坡度分级；客户确认即“公众代表”复核；案例复盘即 lessons-learned
软件研发全流程协同	缺陷修复→测试验证→发布确认全部进折返点；K 标即变更留痕
金融风控与理赔自动化	多源信号聚合→风险核实→处置→合规审计：风险台账 + 不可变账本原生适配

二、核心洞察：铁路的"人字坡"智慧

制度原型：青龙桥"人字形折返"展线

1908 年，詹天佑在八达岭 33‰ 极限坡度上设计"人字形"展线：列车无法直爬陡坡，必须在**折返点停车、换向、以退为进**。这条铁路运行 118 年零重大事故的教训是——**在能力极限前主动停下重估，比硬闯更接近安全与效率的最优解**。

核心洞察：多 Agent 协同系统与陡坡列车同构——任务爬坡（难度/风险升高）时，Agent 的自回归式自动续行就是"硬闯陡坡"。治理的关键不是更强的模型，而是**制度化的折返点**：让系统在关键节点"必停、复核、可回头"。

2.1 四大机制（一句话版）

机制	铁路原型	Agent 工程转译	代码落点
折返点 Switchback Node	到折返点必停	固定检查点，任何一方否决即强制折返	<code>Checkpoint.resolve()</code>
坡度分级 Grade-based Access	33‰ 分陡缓	缓/中/陡三级，越陡复核越严	<code>grade_access()</code>
K 标版本 K-marker	铁路里程标	不可变内容寻址哈希链，永久留痕	<code>KMarkerLedger.append()</code>
道岔三态 Switch States	正线/侧线/入段	状态机，不设自动恢复	<code>governor.switch()</code>

2.2 一句话说服力

对评审的对应

赛道 1 评审点"结果校验、复审、安全熔断审计"，正是本协议的四条不变量：**到站必停**（熔断）、**任何否决即折返**（复审）、**K 标不可篡改**（审计）、**不设自动恢复**（安全）。治理理念即工程机制——这是本方案最直接的叙事桥梁。

三、方案设计：折返治理协议（What）

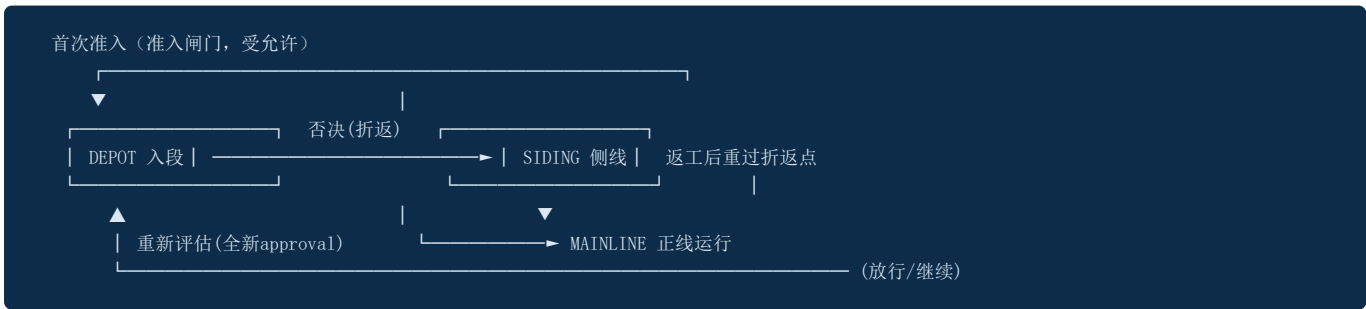
3.1 架构总览



3.2 协议不变量（可复算、可复核、可接手）

- 到站必停**: 每个折返点未裁决前，任务不得越过该点继续执行。
- 任何否决即折返**: 一票 `TURN_BACK` / `DEPOT` ⇒ 整点折返/入段，绝不因多数票放行。
- 坡度越高越严**: 陡坡需三方（责任人/专业/公众）复核，缓坡仅责任人。
- K 标不可篡改**: SHA-256 内容寻址哈希链，篡改即断链。
- 不设自动恢复**: 已被放行、又被拉回检修的任务禁止自动回正线，必须全新复核。
- 票齐才裁决**: 缺票即拒绝裁决，不搞少数服从多数。

3.3 状态机（道岔三态）



关键：不设自动恢复

京张项目 5 轮"内容增强"全部无效（70 是基准、84 是孤例）的教训说明：**自动续行会放大错误**。本协议强制检修态回正线需经折返点重新评估—— `NoAutoResumeError` 是安全熔断的工程化表达。

四、多 Agent 协同设计（以 AgentTeams 为设计基点）

设计基点：AgentTeams（原名 Hiclaw）

按赛道要求，协同设计以 AgentTeams（开源多 Agent OS）为基点：**Manager Agent 协调多个 Worker Agent 在 Matrix 房间中协作，全程对人类可见、可实时干预**；Worker 仅持消费者令牌、不持真实 API Key；经 Higress AI Gateway 安全访问 LLM 与外部 API / MCP Servers / Skills（出处：hiclaw.io、github.com/alibaba/hiclaw）。

4.1 Agent Identity 清单（6 个身份）

ID	角色	类型	职责	AgentTeams 落位
M-OPER	运营总监	Manager	任务接单、坡度拆解、调度、触发折返点	Manager Agent（Control Flow）
W-RESEARCH	调研专员	Worker	情报采集、事实核验、来源管理	Worker + MCP（经 Higress）
W-CREATE	内容专员	Worker	结构化工件生产、双语输出	Worker + Skill
W-VERIFY	校验专员	Worker	数值三处对齐、引用双向、哈希核对、复算	Worker + evidence-verify
W-RISK	风险与合规	Worker	风险台账、权利声明、合规矩阵	Worker + 审计记录
G-GOVERNOR	监理 Agent	治理组件	折返裁决、坡度准入、道岔三态、不设自动恢复	Matrix 房间人机可见、可干预

4.2 闭环 8 步映射

步骤	执行者	机制落点
① 任务输入	M-OPER	Matrix 房间接单，登记任务卡
② 任务拆解	M-OPER	坡度分级（grade-access）→ 子任务 DAG
③ 上下文传递	全 Agent	Matrix 共享上下文 + SharedState（K 标溯源）
④ 工具调用	Worker	经 Higress 调 MCP 工具，凭证透传
⑤ 结果验证	W-VERIFY	evidence-verify：三处对齐/双向引用/哈希
⑥ 执行证据沉淀	G-GOVERNOR	K 标账本 seal（不可变哈希链）
⑦ 审批与回滚	G-GOVERNOR + 人	折返点三方复核 → 放行/折返/入段（不设自动恢复）
⑧ 经验沉淀	全 Agent	lessons-learned 写回 AgentMemory → 教训-规则闭环

4.3 上下文与 RAG（4 选 2，本项目做 3 项）

- 共享状态管理 SharedState：Manager 写、Worker 读，带 K 标版本溯源，杜绝 package_id 残留 式漂移；
- Agent 记忆存储 AgentMemory：追加式情景记忆 + 关键词检索，是知识库 RAG 的种子层，可协议适配替换向量索引；
- 轨迹可观测 Tracer：OTel 形状事件流即执行轨迹。

五、Skill 工程体系与生态复用

六大核心 Skill 全部以可执行 Python 函数实现（switchback/skills.py），每个携带机器可读 SkillSpec（名称/用途/输入输出/调用条件/依赖/失败处理/安全边界/复用价值），可沉淀进 AgentTeams Skill 生态。

Skill	用途	输入→输出	失败处理	复用价值
grade-access	坡度分级准入	task payload → Grade	无法判定默认陡坡（从严）	任务准入层
evidence-verify	证据核验	claims+sources → VerificationReport	任一失败 → 强制折返	防幻觉第一闸门
kmarker-ledger	K 标账本	task+label+payload → KMarker	断链 → 拒绝写入	审计留痕
risk-ledger	风险台账	风险字段 → 条目	score≥4 → 强制人工复核	合规风险视图
switch-decision	折返裁决	Checkpoint → Verdict	缺票 → 拒绝裁决	审批/回滚核心
lessons-learned	经验沉淀	复盘结论 → MemoryEntry	失败不阻断主线	越用越聪明

实证：evidence-verify 的京张检验

京张的 CRLF 哈希错位、数值三处对不齐、引用缺失正是该 Skill 的设计输入；每一版提交在放行前都经其核验。核验失败 → governor.turn_back() → 任务进侧线返工，不自动续行。

六、工具与集成（MCP / 可观测 / RAG）

6.1 MCP 与工具集成契约（推荐项）

采用 AgentTeams 的标准接入：Worker 经 **Higress AI Gateway** 访问外部 API / MCP Servers / Skills，凭证透传 + 消费者令牌。集成契约四要素：

契约	本方案
协议	MCP（经 Higress；必要时自有 JSON 契约等价实现）
鉴权	凭证透传；Worker 不持真实 Key，仅持消费者令牌
输入输出 Schema	Skill 的 SkillSpec 声明（JSON）
错误处理与审计	失败进折返点（折返/入段）；K 标账本 + trace 全程记录

迁移成本判断：Skill 与协议解耦、契约即 JSON，未来迁 MCP 只需协议适配，无需重设计工具调用链。

已交付：.mcp.json 真实 MCP 配置（非文档声称）

仓库根 `.mcp.json` 配置三个 MCP server： `filesystem`（工作区安全访问，对应 AgentTeams MinIO 共享存储）、 `memory`（AgentMemory 跨 Agent 共享检索）、 `github`（拉 PR/CI/check-run，凭证透传）。Skill→MCP 迁移契约见 `docs/mcp-integration.md`。

6.2 可观测（推荐项）

- Trace/Log/Metrics 三类全覆盖：** `Tracer` 输出 OTel 形状 JSONL（SPAN_START/SPAN_END/LOG）+ 指标计数器，满足"建议遵循 OpenTelemetry GenAI 标准"；
- 评估：** 基于观测数据支持复算与逐 K 标核对，量化推理效果与效率。

6.3 推荐的生态工具对齐（不堆叠，说明设计理念与可替换性）

框架层以 AgentTeams 为基点；网关 Higress；可观测与 AI 管理中心（Nacos/AgentLoop 等）按需接入。评审重点在设计理念、接口契约、必要性、可替换性、权限边界与端到端闭环证据，本项目全部以代码与案例给出。

七、安全、风险控制与合规

7.1 权限边界

- 唯一能改变任务状态的是 `G-GOVERNOR` ； Worker 只能产出工件与复核结果，不能自我放行；
- 凭证不落 Worker（消费者令牌模型）；人工在 Matrix 房间内行使"终审/公众代表"职权。

7.2 风险台账（结构化，score≥4 强制人工复核）

风险	评分	缓释	人工复核
Agent 数值幻觉与引用错位	4	evidence-verify 强制三处对齐 + 双向引用 + 内容哈希	专业复核
长会话状态漂移	4	K 标版本 + SharedState 溯源（阻断 package_id 残留）	运营 Agent 复核
第三方素材权利边界不清	3	rights-ledger 逐资产登记 + 许可证检查	法务复核

7.3 审计与回滚

K 标账本（不可变哈希链）+ OTel 兼容 trace 构成端到端审计链路：**谁、何时、基于哪个 K 标、结论的哈希** 全部可复核；回滚即"侧线折返 / 入段检修"，且不设自动恢复。

八、工程落地、运行验证与可复现

可执行代码包（初赛可选·本项目已交付）

github.com/sukikeeling/switchback: Python 3.10+, 零第三方运行依赖, `pip install -e .` 即跑; 43 项 `pytest` 全绿 (含状态机边界、哈希链冲突、持久化往返、CLI 审查修复); GitHub Actions CI 覆盖 Python 3.10/3.11/3.12。

交付门禁（借签 TRIO3.0-oss）：8 维度全系统验证

`bash scripts/full-system-verify.sh` —— 代码质量 / 协议不变量 / 安全熔断 / K标账本 / Skill 声明 / CLI 幸福路径 / 案例可复现 / 诚实局限声明, 任一失败 **exit 1 阻断交付**。把 `evidence-verify` 从“返回 False”升级为“阻断交付”。

8.1 一键复现旗舰案例（真实输出）

```
$ python -m switchback.cli replay jingzhang
```

== 京张 84 分案例重放 (Switchback Governance in action) ==

版本	PR	分	裁决	证据
v5	PR#605	67	PASS	✓
v8	PR#1220	70	折返↔	✓
v8.1	PR#1468	84	PASS	✓
v8.2	PR#1816	70	折返↔	✓
v8.5	PR#2205	77	折返↔	✓
v8.10	PR#2328	76	折返↔	✓

最高分: 84 (v8.1 PR#1468) → 84 高水位保持

教训: '加内容' 5 轮无效: 克制 + 结构化证据 + 折返复核 = 高分配方。

每次提交都过“证据核验 → 折返点复核 → K 标放行/折返”, 账本不可变、Trace 可回放。

真实任务闭环

结果校验

安全熔断审计

--live 模式: GitHub API 验证 PR 真实存在（非剧本）

```
$ python -m switchback.cli replay jingzhang --live
{
  "repo": "open-city-ai/haidian",
  "checked": [
    { "version": "v5", "pr": 605, "exists": true, "merged": true, "title": "提交方案 v5: 结构化证据层升级",
    { "version": "v8.1", "pr": 1468, "exists": true, "merged": true, "title": "提交方案 v8.1: ...P0 修复",
    ...
  ]
}
```

6 个 PR 全部经 GitHub API 验证为真实存在, 标题/合并状态/分数对得上——剧本不再是“写死的”, 是“经 API 验证的真实记录”。

8.2 跨行业第二案例：运维事故自主闭环（已实现，非承诺）

```
$ python -m switchback.cli replay ops
```

== 运维事故自主闭环 (Switchback Governance · 跨行业复用) ==
事故 INC-2026-0813: 支付服务 5xx 飙升

阶段	裁决	坡度	证据
alert-triage	PASS	medium	✓
fix-v1	折返↔	steep	✓
fix-v2	PASS	medium	✓
recovery-verify	PASS	medium	✓
postmortem	PASS	gentle	✓

关键：fix-v1 因回滚风险被责任人一票否决→侧线折返；fix-v2 金丝雀方案三方通过放行。
折返 1 次，放行 4 次，全程不设自动恢复。
跨行业复用实证：同一套折返治理协议，从城市设计评审到生产事故运维。

同一套协议、同一套 Skill，无需改代码即可治理 **城市设计评审** **零人工运维** 两类场景——"场景可复制性"实证而非口号。

8.3 命令行全流程（评审可直接运行）

```
$ python -m switchback.cli init
$ python -m switchback.cli register jz-001 --title "京张方案 v8.1" --grade steep
$ python -m switchback.cli verify jz-001 --claims claims.json --sources sources.json
$ python -m switchback.cli approve jz-001 --label release
$ python -m switchback.cli status jz-001
$ python -m switchback.cli ledger # 输出 K 标哈希链并校验完整性
```

8.4 测试证据（38 项）

- `test_protocol.py`（14）：折返裁决 / 一票否决 / 不设自动恢复 / 首次放行例外 / 账本篡改检测；
- `test_boundaries.py`（15）：**状态机边界**（侧线返工回正线 / 多轮折返累积 / 全 DEPOT 票 / 混合票优先级 / 被放行又入段禁止自动恢复）、**哈希链冲突**（空账本 / 篡改 payload / 伪造 sha256 / 断链 prev / 持久化往返一致）、**任务卡持久化往返**；
- `test_skills.py`（8）：六 Skill 输入输出与组合；
- `test_jingzhang_case.py` + `test_ops_case`：两案例重放断言真实裁决结果。

九、可行性与落地计划

9.1 落地路线

阶段	内容	证据
已实现	折返治理协议 + 六 Skill + 双案例（京张/运维）+ 可观测	38 项测试（含状态机边界/哈希链冲突） / CI 全绿 / 双案例复现输出
初赛→复赛 (9.3)	AgentTeams 集群实际部署（Docker + Matrix 房间 + Higress）、Demo 视频	可执行代码包 → 运行 Demo
复赛→决赛 (9.22)	评测报告、生态 Skill 分发、第三案例（客服/研发）	现场 Demo + 审计链路展示

9.2 治理协议跨行业复制（场景首发计划）

协议与业务解耦：换 Skill、换 MCP 工具即可迁移。四个参考方向均已给出落点（见 1.3），最优先落地“零人工运维”（与京张“提交-评审-修复”闭环同构）。

9.3 风险与应对（本方案自身的风险台账）

风险	应对
治理协议被误读为“流程审批系统”	定位为“人机监理”；强调折返点的制度原型与可复算不变量
复赛 AgentTeams 部署复杂度	协议与框架解耦，先容器化单机 demo，再上集群
零依赖范围限制（无真实 LLM 调用）	初赛以方案设计为主；复赛接入 AgentTeams + LLM 补全可运行 Demo

十、开放 / 开源计划

- **协议**：Apache-2.0，全量开源（与 AgentTeams/Hiclaw 一致）；
- **仓库**：github.com/sukikeeling/switchback —— README（中英）、协议规范、架构、Agent Identity、Skill 清单、CI、SECURITY、CONTRIBUTING 齐备；
- **数据与权利**：京张案例数据来自公开开源竞赛仓库，demo 不含非公开/受限数据；逐资产权利登记（rights-ledger）；
- **依赖边界**：运行零第三方依赖；开发依赖仅 pytest——第三方依赖与 API 调用边界清晰可审计；
- **生态**：Skill 的 SkillSpec 可沉淀进 AgentTeams Skill 门户 / AI 市场，向社区开放复用。

开源贡献度自评

一个零依赖、可复现的治理协议实现 + 一段真实竞赛闭环证据 + 一份把"治理理念翻译成工程机制"的完整文档集。评审可直接 clone、跑测试、看账本、复现案例。

附录：赛道 1 评审指标对照表

评审维度	权重	本方案落点（可点击核实）
场景价值与行业可复制性	25%	一、三、九：京张 84 分真实闭环 + 四方向复制
多 Agent 协同与自主闭环能力	25%	四：AgentTeams 基点、6 Agent、闭环 8 步、上下文 3 项
Skill 工程体系与生态复用	25%	五：六大可执行 Skill + SkillSpec 机器声明
工程落地、运行验证与安全可审计	20%	七、八：可复现代码包、23 测试、K 标账本、风险台账、不设自动恢复
开放/开源贡献	5%	十：Apache-2.0 全量开源、依赖边界清晰

初赛必交内容核对

要求	本方案
作品简介 ≤500 字	competition/intro-500.md（项目名 ~20 字）
方案 PPT/PDF（场景/设计/Skill/可行性）	本 PDF 一至十章
Agent 分工 / 任务拆解 / 上下文传递	四、4.2 闭环 8 步
结果验证 / 异常分支 / 安全边界 / 风险控制	五、七（evidence-verify / 道岔 / 风险台账）
开放/开源计划	十
AgentTeams 设计基点 + Agent Identity 清单	四、docs/identity.md（附录A 对应）
可执行代码包（可选）	已交付：github.com/sukikeeling/switchback